FASTAPI CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & RUN

Installation

```
pip install fastapi uvicorn[standard]
from fastapi import FastAPI
app = FastAPI(title='My API', version='1.0.0')
uvicorn main:app --reload
uvicorn main:app --host 0.0.0.0 --port 8000
```

Docs: /docs (Swagger) and /redoc (ReDoc)

02. PATH OPERATIONS

Architecture

```
@app.get('/items', response_model=List[Item])
async def list_items():
return await db.items.find_all()
@app.post('/items', status_code=201)
async def create(item: Item):
return await db.items.insert(item)
```

HTTP verbs: get post put patch delete head options

03. PATH & QUERY PARAMS

YAML / Config

```
@app.get('/users/{user_id}')
async def get_user(user_id: int):
return {'id': user_id}
@app.get('/search')
async def search(q: str, limit: int = 10, ski...
...
```

Type annotations auto-validate and document

04. REQUEST BODY (PYDANTIC)

Workflow

```
from pydantic import BaseModel, EmailStr
class User(BaseModel):
name: str
email: EmailStr
age: int = Field(ge=0, le=120)
@app.post('/users')
async def create(user: User): ...
```

Automatic validation + OpenAPI schema generation

05. DEPENDENCY INJECTION

CRUD API

```
from fastapi import Depends
def get_db() -> Generator:
db = SessionLocal()
try: yield db
finally: db.close()
async def get_current_user(token=Depends(oaut...
...
def endpoint(db=Depends(get_db), user=Depends...
```

06. RESPONSE MODELS

Integration

```
class UserOut(BaseModel):
id: int
name: str
# no password field
@app.get('/users/{id}', response_model=UserOu...
async def get_user(id: int): ...
response_model_exclude_unset=True  # skip def...
response_model_exclude={'password'}
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. AUTH (OAUTH2 + JWT)

Hardening

```
from fastapi.security import OAuth2PasswordBe...
oauth2_scheme = OAuth2PasswordBearer(tokenUrl...
@app.post('/token')
async def login(form: OAuth2PasswordRequestFo...
user = authenticate(form.username, form.passw...
token = create_jwt(user.id)
return {'access_token': token, 'token_type': ...
```

08. MIDDLEWARE & CORS

Observability

```
from fastapi.middleware.cors import CORSMiddl...
app.add_middleware(CORSMiddleware,
allow_origins=['https://myapp.com'],
allow_methods=['*'],
allow_headers=['*'])
@app.middleware('http')
async def add_header(request, call_next):
response = await call_next(request)
response.headers['X-Process-Time'] = str(t)
return response
```

09. ASYNC ENDPOINTS

Optimization

```
@app.get('/slow')
async def slow_endpoint():
result = await slow_io_operation()
return result
import asyncio
await asyncio.gather(task1(), task2())
```

Use sync def for CPU-bound (runs in threadpool)
Use async def for I/O-bound operations

10. BACKGROUND TASKS

Unit Tests

```
from fastapi import BackgroundTasks
@app.post('/notify')
def notify(bg: BackgroundTasks, email: str):
bg.add_task(send_email, email, 'Welcome!')
return {'message': 'queued'}
```

For heavy work: use Celery + Redis instead

11. TESTING

Commands

```
from fastapi.testclient import TestClient
client = TestClient(app)
def test_create():
resp = client.post('/users', json={'name':'Al...
assert resp.status_code == 201
assert resp.json()['name'] == 'Alice'
```

Override deps: app.dependency_overrides[get_db] = mock_db

12. COMMON GOTCHAS

Warning

async def needs await forgetting await blocks event loop
Pydantic V2 breaking changes vs V1 (field vs Field)
Don't use sync I/O inside async endpoints
global state is shared across requests use Depends
Response model strips extra fields can miss data